

EXPERIMENT 2

Aim: Create a Blockchain using Python

Theory :

A **blockchain** is a decentralized and distributed digital ledger that stores data in the form of interconnected blocks. Each block contains a set of transactions or data, a timestamp, the hash of the previous block, and its own unique hash. By linking each block to the previous one through cryptographic hashes, blockchain ensures that the stored information is secure, transparent, and resistant to tampering.

In Python, a blockchain can be implemented using object-oriented programming by creating a Block class and a Blockchain class. The Block class stores the block's data, timestamp, previous hash, and calculates its own hash using a cryptographic hashing algorithm such as SHA-256. The Blockchain class manages the chain of blocks, beginning with a **genesis block**, and provides methods to add new blocks and verify the integrity of the chain.

The hash of each block is generated from its contents, making every block unique. If any information within a block is modified, its hash changes, which breaks the connection with subsequent blocks. This feature ensures **data integrity** and makes unauthorized modifications easily detectable.

The blockchain follows these fundamental principles:

- **Decentralization:** Data is distributed across multiple nodes rather than stored in a central location.
- **Immutability:** Once a block is added to the chain, it cannot be altered without changing all subsequent blocks.
- **Transparency:** Every participant can verify the contents and integrity of the blockchain.
- **Security:** Cryptographic hashing protects the data from unauthorized changes.

In this experiment, Python is used to simulate the basic functioning of a blockchain. Blocks are created, linked together using hashes, and added sequentially to form a chain. This demonstrates the core concepts of blockchain technology, including block creation, hash generation, chain validation, and secure data storage.

Task Performed :

Step 1 — Install & Import Required Libraries

PROGRAM :

```
!pip install flask==2.2.5 flask-ngrok pyngrok -q
```

```
import datetime
import hashlib
import json
```

```
class Blockchain:
```

```
    def __init__(self):
```

```
        self.chain = []
```

```

self.transactions = []

self.create_block(proof=1, previous_hash='0')

def create_transactions(self, sender, receiver, amount):
    """
    Adds a new transaction to the list of pending transactions.
    These transactions will be included in the NEXT mined block.
    Returns the index of the block that will hold this transaction.
    """
    self.transactions.append({
        'sender': sender,
        'receiver': receiver,
        'amount': amount
    })

    previous_block = self.get_previous_block()

    return previous_block['index'] + 1

def create_block(self, proof, previous_hash):
    """
    Creates a new block and adds it to the chain.
    Each block contains:
    - index
    - timestamp
    - proof (PoW output)
    - previous block hash
    - transactions (the pending transactions being mined into this block)

    After the block is created, the pending transactions list is
    cleared.
    """
    block = {

```

```

        'index': len(self.chain) + 1,

        'timestamp': str(datetime.datetime.now()),

        'proof': proof,

        'previous_hash': previous_hash,

        'transactions': self.transactions
    }

```

```

self.chain.append(block)

self.transactions = []

return block

```

```

def get_previous_block(self):

    """Returns the last block in the chain."""

    return self.chain[-1]

```

```

def proof_of_work(self, previous_proof):

    """

```

Proof of Work (PoW) algorithm:

- Find a number (new_proof), the "golden nonce"
- Such that the SHA256 hash of (new_proof^2 - previous_proof^2) starts with '000' (3 leading zeros)

```

    """

    new_proof = 1

    check_proof = False

    while check_proof is False:

        hash_operation = hashlib.sha256(

            str(new_proof**2 - previous_proof**2).encode()

        ).hexdigest()

```

```

        if hash_operation[:3] == '000':

            check_proof = True

        else:

            new_proof += 1

    return new_proof

def hash(self, block):

    """Creates a SHA256 hash of a block (sorted keys for consistent
    hashing)."""

    encoded_block = json.dumps(block, sort_keys=True).encode()

    return hashlib.sha256(encoded_block).hexdigest()

def is_chain_valid(self, chain):

    """

    Validates the blockchain by checking:

    1. previous_hash matches the actual hash of the previous block.

    2. The PoW condition (hash starting with '000') holds for each block.

    """

    previous_block = chain[0]

    block_index = 1

    while block_index < len(chain):

        block = chain[block_index]

        if block['previous_hash'] != self.hash(previous_block):

            return False

        previous_proof = previous_block['proof']

        proof = block['proof']

        hash_operation = hashlib.sha256(

            str(proof**2 - previous_proof**2).encode()

        ).hexdigest()

```

```

        if hash_operation[:3] != '000':

            return False

        previous_block = block

        block_index += 1

    return True

def mine_pending_transactions(self):
    """
    Mines a new block ONLY IF there is at least one pending transaction.

    Returns the mined block, or None if there was nothing to mine.
    """

    if len(self.transactions) == 0:

        return None

    previous_block = self.get_previous_block()

    previous_proof = previous_block['proof']

    proof = self.proof_of_work(previous_proof)

    previous_hash = self.hash(previous_block)

    block = self.create_block(proof, previous_hash)

    return block

```

Step 2 - Quick Test of the Blockchain Logic (no Flask yet)

PROGRAM :

```

blockchain = Blockchain()

result = blockchain.mine_pending_transactions()
print('Mining with empty pool ->', result)

idx = blockchain.create_transactions('Alice', 'Bob', 10)
blockchain.create_transactions('Bob', 'Charlie', 5)
print('Pending transactions:', blockchain.transactions)
print('These will go into block index:', idx)

```

```

block = blockchain.mine_pending_transactions()
print('Mined block:', block)

print('Pending after mining:', blockchain.transactions)

print('Chain valid?', blockchain.is_chain_valid(blockchain.chain))

```

OUTPUT:

Mining with empty pool -> None

Pending transactions: [{'sender': 'Alice', 'receiver': 'Bob', 'amount': 10}, {'sender': 'Bob', 'receiver': 'Charlie', 'amount': 5}]

These will go into block index: 2

Mined block: {'index': 2, 'timestamp': '2026-09-02 06:36:44.078629', 'proof': 533, 'previous_hash': 'c0c303f743620dd278f9c7a2a76f53796179786bf07313f2ecf0147b278d916d', 'transactions': [{'sender': 'Alice', 'receiver': 'Bob', 'amount': 10}, {'sender': 'Bob', 'receiver': 'Charlie', 'amount': 5}]}

Pending after mining: []

Chain valid? True

PROGRAM:

```

from flask import Flask, jsonify, request

app = Flask(__name__)

blockchain = Blockchain()

@app.route('/', methods=['GET'])

def home():

    return "<h1>Blockchain Server is Running</h1><p>Use <b>/get_chain</b> to view the blockchain.</p>"

@app.route('/add_transaction', methods=['POST'])

def add_transaction():

    json_body = request.get_json(force=True)

    required = ['sender', 'receiver', 'amount']

    if not all(key in json_body for key in required):

```

```

        return 'Missing fields in transaction', 400

index = blockchain.create_transactions(

    json_body['sender'], json_body['receiver'], json_body['amount']

)

    response = {'message': f'This transaction will be added to Block
{index}'}

    return jsonify(response), 201


@app.route('/mine_block', methods=['GET'])

def mine_block():

    if len(blockchain.transactions) == 0:

        response = {'message': 'No transactions to mine. Add a transaction
first.'}

        return jsonify(response), 400

previous_block = blockchain.get_previous_block()

previous_proof = previous_block['proof']

proof = blockchain.proof_of_work(previous_proof)

previous_hash = blockchain.hash(previous_block)

block = blockchain.create_block(proof, previous_hash)

response = {

    'message': 'Congratulations, you just mined a block!',

    'index': block['index'],

    'timestamp': block['timestamp'],

    'proof': block['proof'],

    'previous_hash': block['previous_hash'],

    'transactions': block['transactions']

}

    return jsonify(response), 200

```

```

@app.route('/get_chain', methods=['GET'])
def get_chain():
    response = {
        'chain': blockchain.chain,
        'length': len(blockchain.chain)
    }
    return jsonify(response), 200

@app.route('/is_valid', methods=['GET'])
def is_valid():
    valid = blockchain.is_chain_valid(blockchain.chain)
    if valid:
        response = {'message': 'All good. The Blockchain is valid.'}
    else:
        response = {'message': 'Houston, we have a problem. The Blockchain is not valid.'}
    return jsonify(response), 200

```

Step 3 - Run the server

PROGRAM :

```

from pyngrok import ngrok
import os
from threading import Thread
# 1. Kill any existing tunnels to start fresh
ngrok.kill()
# 2. Set your auth token
ngrok.set_auth_token("3I7gMEFA6xM91oLVRnsjXhhJW5p_mbYWNSAcxbEUHNDe5bx9")
# 3. Create a new tunnel

```

```

public_url = ngrok.connect(5000)
print(' * Public Postman URL:', public_url.public_url)
print(' * Click "Visit Site" if you see a browser warning.')
# 4. Define a function to run the Flask app
def run_flask_app():
    app.run(port=5000, use_reloader=False)
# 5. Start the Flask server in a separate thread
flask_thread = Thread(target=run_flask_app)
flask_thread.start()

print(' * Flask server is running in a background thread.')
print(' * To stop the server, you will need to interrupt the kernel
(Runtime > Interrupt execution).')

```

OUTPUT:

```

* Public Postman URL: https://buffoon-sixfold-arbitrate.ngrok-free.dev
* Click "Visit Site" if you see a browser warning.
* Serving Flask app '__main__'
* Flask server is running in a background thread.
* To stop the server, you will need to interrupt the kernel (Runtime >
Interrupt execution).
* Debug mode: off
INFO:werkzeug:WARNING: This is a development server. Do not use it in a
production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000

```

PROGRAM:

```

from pyngrok import ngrok
tunnels = ngrok.get_tunnels()
for tunnel in tunnels:
print(f"Active Tunnel: {tunnel.public_url} -> {tunnel.config['addr']}")

```

OUTPUT:

```

Active Tunnel: https://buffoon-sixfold-arbitrate.ngrok-free.dev ->
http://localhost:5000

```

Blockchain Server is Running

Use `/get_chain` to view the blockchain.

```
buffoon-sixfold-arbitrate.ngrok-free.dev/get_chain
Pretty-print ☐
{"chain":[{"index":1,"previous_hash":"0","proof":1,"timestamp":"2026-09-02 06:36:49.316763","transactions":[]},"length":1}
```

```
buffoon-sixfold-arbitrate.ngrok-free.dev/mine_block
Pretty-print ☐
{"message":"No transactions to mine. Add a transaction first."}
```

```
buffoon-sixfold-arbitrate.ngrok-free.dev/is_valid
Pretty-print ☐
{"message":"All good. The Blockchain is valid."}
```

Google Colab File Link : [🔗 Blockchain Experiment 2 NEW.ipynb](#)

Conclusion :

In this experiment, a basic blockchain was successfully created and implemented using Python. The experiment demonstrated the creation of blocks, addition of transactions, generation of SHA-256 hashes, Proof of Work, and validation of the blockchain. The blockchain was also integrated with a Flask web server, allowing transactions to be added, blocks to be mined, the chain to be viewed, and its validity to be verified through API endpoints. The successful validation result shows how cryptographic hashing and Proof of Work help maintain data integrity, security, and resistance to tampering in a blockchain system.